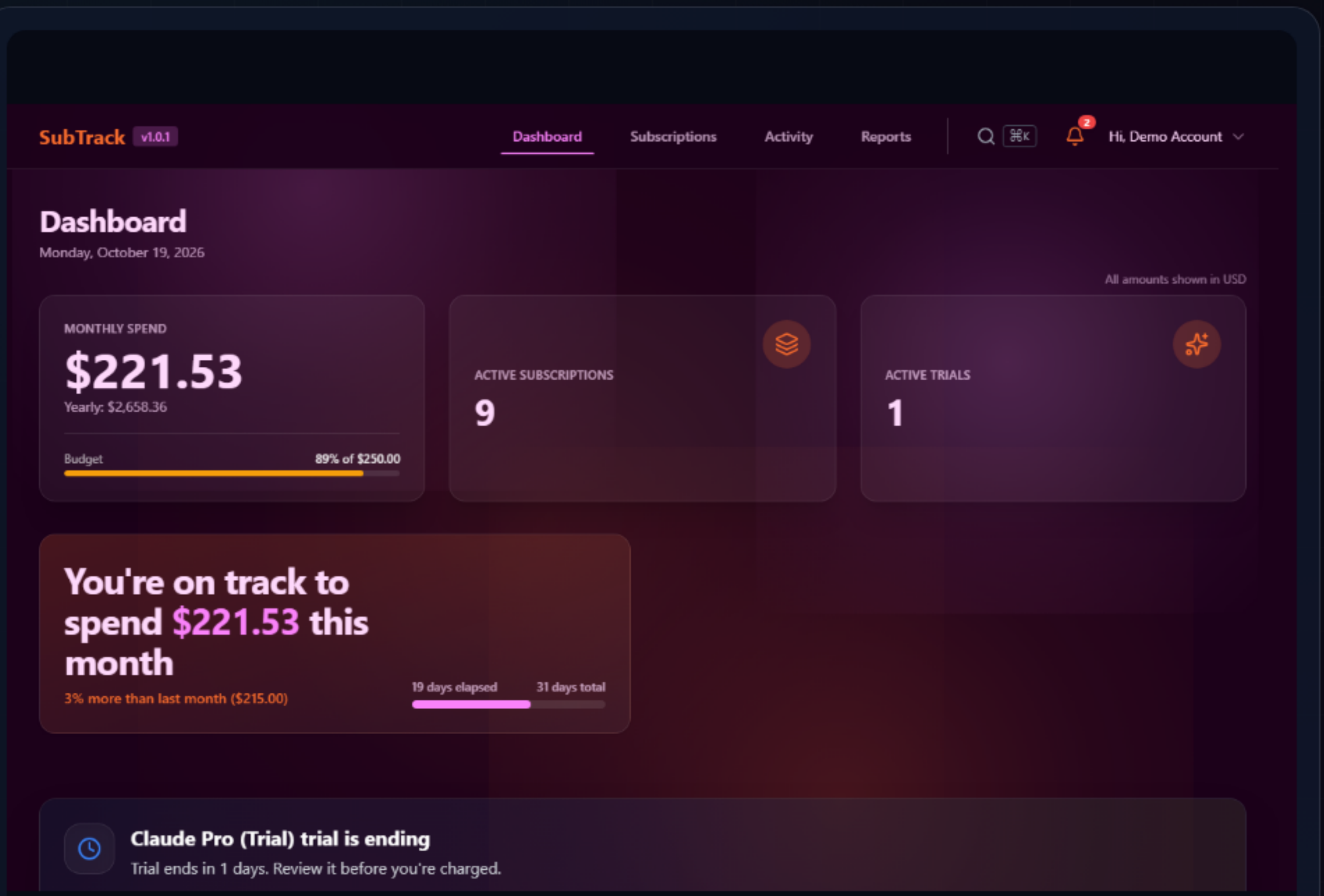
I built a subscription tracker. The real project was how it runs.

SubTrack: Building a Production-Style DevOps Workflow by Aditya

✓ 0 failed requests during a live rollout test



SubTrack Live Dashboard: Active spend tracking and automated renewal monitoring.

The Big Picture: Cloud-Native Architecture

Zero-downtime rollouts • Declarative GitOps

SubTrack Architecture

6-Box Core Flow

Kubernetes (kind) • 15 Managed Resources

01 SOURCE CONTROL
Git Repository

Single source of truth for both application code and Kubernetes declarative manifests.

`github.com/Aditya-k-11/sub_tracker`

02 AUTOMATED GATES
GitHub Actions CI

Automated linting, image build with Buildx caching, security verification, and Docker Hub tag & push.

`Run #48 • 1m 6s duration`

03 DECLARATIVE CD
ArgoCD Controller

GitOps reconciliation engine with automated sync, prune, and drift correction across the cluster.

`15 / 15 Resources Synced & Healthy`

04 TRAFFIC INGRESS
NGINX Ingress

L7 edge routing, TLS termination, and path-based routing for SPA frontend and backend API.

`subtrack.local • HTTPS / HTTP`

05 SCALABLE RUNTIME
Backend Service

Node.js Express API with HPA autoscaling (3 to 6 pods), PodDisruptionBudget, and readiness probes.

`0.00% failed in 3,989-req rollout`

06 DATA LAYER
MongoDB (v7)

Stateful document store backed by PersistentVolumeClaim with automated nightly mongodump CronJob.

`mongo-data-pvc • Standard StorageClass`

Pipeline: Git Push → CI Build & Push → ArgoCD GitOps → Ingress → HPA Pods (3→6) → MongoDB v7

Zero-downtime rollouts architecture

6-Box Flow: Ingress routing, GitOps reconciliation, auto-scaled backend runtime (3–6 pods), and MongoDB persistence.

Built and tested on a local Kubernetes (kind) cluster

SWIPE →

Every push is built and published automatically

✓ 1m 6s Clean Pipeline Run (Run #48)

✓ Fix ConfirmDialog props causing stuck modal #48

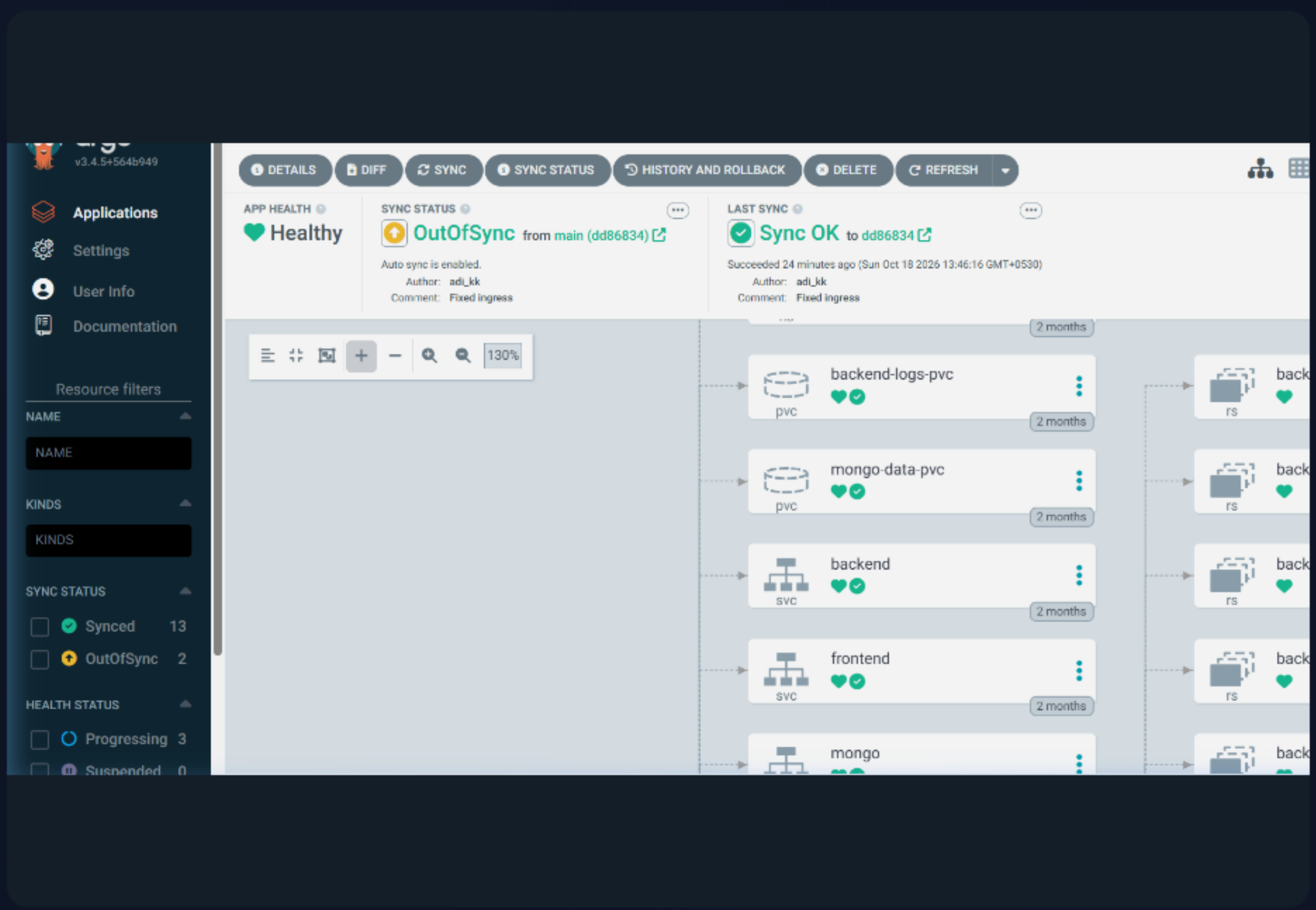
TRIGGER	STATUS	TOTAL DURATION	WORKFLOW
Aditya-k-11 3bb978d main	Success	1m 6s	ci-cd.yaml (push)

build-and-deploy 1m 2s — ✓ Checkout → Docker Buildx → Tag & Push

> **Workflow Run #48:** Triggered on push to main (commit 3bb978d). Builds and pushes Docker images to Docker Hub in 1m 6s.

Git is the single source of truth

✓ ArgoCD Drift Detection: App Healthy, 15 Resources



Declarative GitOps: ArgoCD continuously reconciles cluster state against Git manifests. OutOfSync status flags local manifest divergence on 2 of 15 resources, ensuring automated drift detection.

Making Deployments More Reliable

✓ 3,989 requests during rollout • 0 failed requests

k6 run load-tests/zero-downtime-check.js

```
~/subtrack $ k6 run --vus 10 --duration 90s load-tests/zero-downtime-check.js
```

3,989	0.00%	3.23 ms	6.71 ms
TOTAL REQUESTS	FAILURE RATE	MEDIAN LATENCY (P50)	TAIL LATENCY (P95)

✓ status is 200	100.00% (3,989 passes / 0 failures)
✓ response time < 500ms	99.57% (3,972 passes / 17 failures)
http_req_duration	avg=25.27ms med=3.23ms p(90)=5.06ms p(95)=6.71ms
http_req_failed	0.00% (✓ 0 failed, ✓ 3,989 succeeded)
throughput	44.22 req/s (10 concurrent VUs over 90s rollout)

ℹ **Rollout Observation:** 17 of 3,989 (0.43%) took over 500 ms during the pod swap, none failed.

Zero Failed Requests: 0 HTTP failures out of 3,989 requests during rollout. 17 requests (0.43%) exceeded 500 ms during pod termination and readiness swap. p95 latency: 6.71 ms.

Scaling with Kubernetes

HPA Autoscale: 3 to 6 Pods (258% Peak CPU)

kubectl get hpa backend-hpa -n subtrack --watch

```
~/subtrack $ kubectl get hpa backend-hpa -n subtrack --watch
```

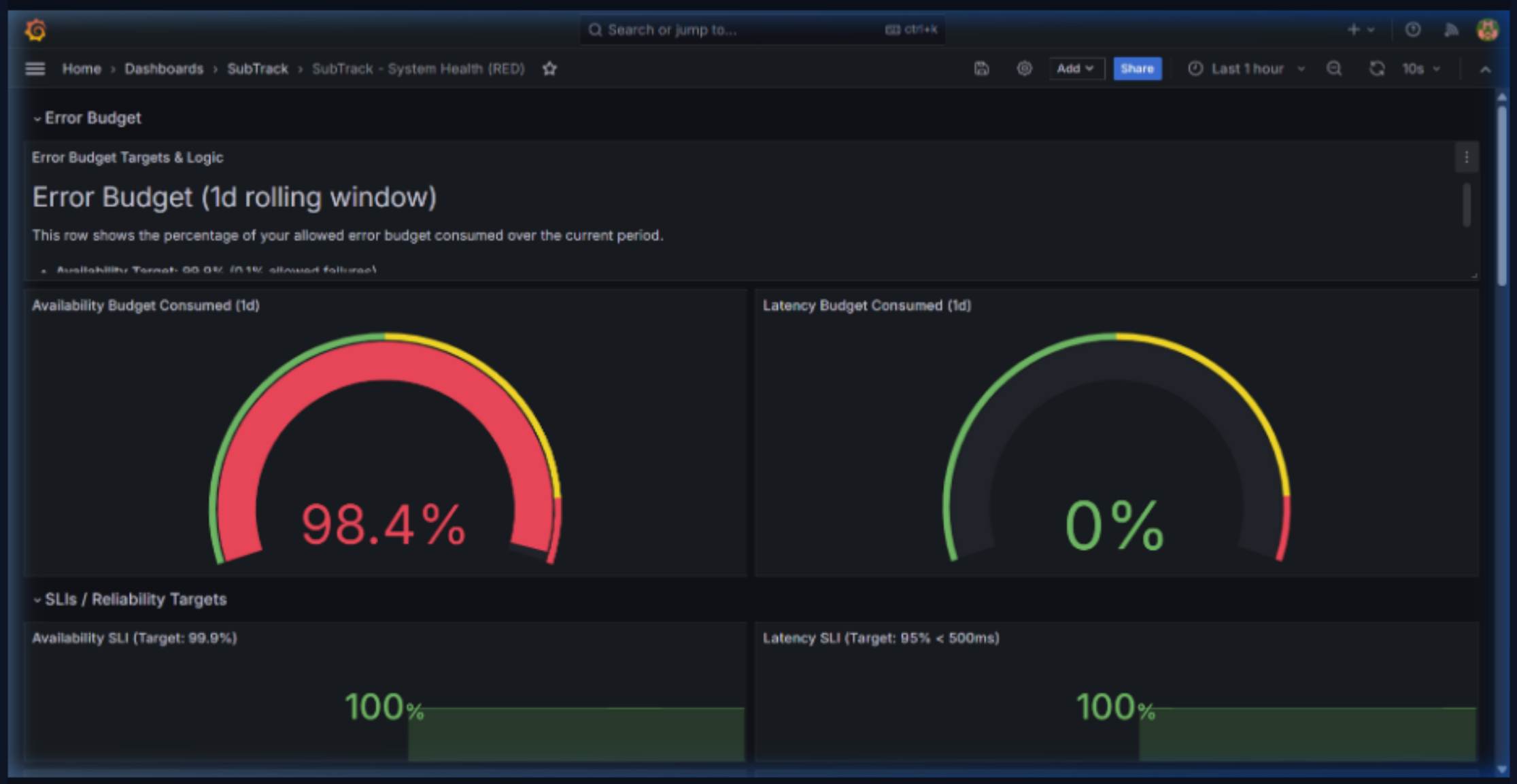
NAME	REFERENCE	TARGETS	MIN	MAX	REPLICAS	HPA EVENT
backend-hpa	Deployment/backend	cpu: 28%/70%	2	6	3	BASELINE LOAD (3 PODS)
backend-hpa	Deployment/backend	cpu: 102%/70%	2	6	3	TARGET EXCEEDED (>70%)
backend-hpa	Deployment/backend	cpu: 161%/70%	2	6	4	SCALED: 3 → 4 PODS
backend-hpa	Deployment/backend	cpu: 197%/70%	2	6	5	SCALED: 4 → 5 PODS
backend-hpa	Deployment/backend	cpu: 258%/70%	2	6	6	PEAK REACHED (6/6 PODS)

3 → 6	258%	2 → 6
OBSERVED SCALING	PEAK CPU (TARGET 70%)	CONFIGURED HPA RANGE

Autoscaling in Action: Configured HPA (min: 2, max: 6, target: 70% CPU). Under k6 load surge, active replicas scaled from 3 to 6 as CPU utilization reached 258%.

Know what is happening inside the system

Latency SLO: 95% < 500 ms • Error Budget: 1-Day Window



Error Budget Monitoring: Grafana RED dashboard tracking Availability (98.4% consumed in 1d rolling window via chaos testing) and Latency (0% budget consumed).

Security Controls & Verification

UID 1000 Non-Root • Bitnami CRD In Cluster • Git Secret Hygiene

Security Controls & Verification — Runtime, Cluster & Repository

```
~/subtrack $ kubectl exec deploy/backend -n subtrack -- whoami && id
node
uid=1000(node) gid=1000(node) groups=1000(node)

~/subtrack $ kubectl get crd sealedsecrets.bitnami.com
NAME                          CREATED AT
sealedsecrets.bitnami.com     2026-08-12T12:20:19Z

~/subtrack $ kubectl get pod -n kube-system -l name=sealed-secrets-controller
NAME                                      READY   STATUS
sealed-secrets-controller-549fdd6dc6-5n4gq  1/1     Running

~/subtrack $ git check-ignore server/.env client/.env
server/.env
client/.env
```

OBSERVED: UID 1000 (NON-ROOT)

BITNAMI CRD REGISTERED

CONTROLLER ACTIVE (1/1)

LOCAL SECRETS EXCLUDED FROM GIT

- ✓ Backend process verified running as unprivileged UID 1000 (node).
- ✓ Bitnami Sealed Secrets controller and CRD active in cluster.
- ✓ Sensitive environment files excluded from active Git tracking.

What I learned

1. Reliability must be measured, not assumed.

2. Automation makes repeatable operations possible.

3. Evidence matters more than claims.

Repository link in the comments

Full manifests, load tests, and observability setup

